HARTA UNUI CALCULATOR

O singură mașină. Două moduri de a-i cere ceva. Un rezultat vizibil.

SCANEAZA

TU
utilizator / student

APEȘI / DAI CLICK
INTERFAȚA GRAFICĂ (GUI)
buton, icon, browser

SCRII O COMANDĂ
TERMINAL + SHELL
comenzi tastate

PROGRAM / APLICAȚIE
instrucțiuni care produc efecte

SISTEMUL DE OPERARE
Windows · Linux · macOS · Android

CPU + RAM
execută și țin datele de lucru

FIȘIERE / SSD
păstrează rezultatele salvate

REȚEA / SERVER
trimite sau primește date

DE CE ÎNVĂȚĂM STRUCTURI DE DATE?

STRUCTURI = ORGANIZARE
Unde punem informația?

ALGORITMI = PAȘI
Ce facem cu informația?

BIG-O = CREȘTERE
Câtă muncă apare în plus?

ȚINE MINTE

De la un click sau o comandă până la un rezultat: DATE + INSTRUCȚIUNI + EXECUȚIE.

Ține minte: terminalul și browserul nu sunt alte calculatoare; sunt două aplicații care pot folosi aceleași resurse.

CALCULATORUL, EXPLICAT

Un telefon, un laptop și un PC au aceeași idee de bază.

SCANEAZA

CUM SE ÎNTÂMPLĂ • URMĂREȘTE SĂGEȚILE

01 APEȘI UN BUTON → 02 APLICAȚIA CERE → 03 SISTEMUL AJUTĂ → 04 MAȘINA LUCREAZĂ

PATRU IDEI DE REȚINUT

1 HARDWARE = piesele fizice

Ce poți atinge: ecran, procesor, memorie, stocare. Procesorul (CPU) execută instrucțiuni.

Imagine: bucătăria și aparatele din ea.

2 SOFTWARE = instrucțiunile

Aplicațiile sunt programe: browser, joc, cameră. Codul le spune ce acțiuni să execute.

Imagine: rețeta pe care o urmează bucătarul.

3 OS = coordonatorul

Windows, Linux, macOS și Android sunt sisteme de operare. Ele gestionează aplicații, fișiere și dispozitive.

OS nu este browserul. Browserul este o aplicație.

4 INTRARE → PRELUCRARE → IEȘIRE

Intrare: clic sau tastă. Prelucrare: se execută instrucțiuni. Ieșire: imagine, sunet, fișier salvat.

Exemplu: apăs volumul → se schimbă sunetul.

ACTIVITATE FĂRĂ CALCULATOR

Un student este utilizatorul, unul aplicația, unul OS, unul procesorul. Simulați apăsarea „Play”.

O FRAZĂ DE ȚINUT MINTE

ECRANUL arată rezultatul; în spate lucrează aplicații, OS și hardware.

Întrebare: Android este hardware, aplicație sau sistem de operare?

BROWSERUL ȘI INTERNETUL

De ce un site nu este „în” browserul tău?

SCANEAZA

CUM SE ÎNTÂMPLĂ • URMĂREȘTE SĂGEȚILE

01 SCRII ADRESA → 02 SE AFLĂ SERVERUL → 03 SE CER FIȘIERELE → 04 SE ARATĂ PAGINA

PATRU IDEI DE REȚINUT

1 BROWSER ≠ INTERNET

Chrome, Firefox și Safari sunt aplicații. Internetul este rețeaua care leagă dispozitive. Webul este un serviciu folosit prin internet.

Browserul este fereastra; internetul este drumul.

2 ADRESĂ ≠ CĂUTARE

O adresă web (URL) arată unde ceri o resursă. Un motor de căutare, precum Google, îți găsește posibile adrese.

site.ro/curs este adresă; „curs Python” este căutare.

3 CINE TRIMITE PAGINA?

Browserul (clientul) cere date unui server. DNS ajută la găsirea adresei lui. Serverul răspunde; browserul afișează.

404 = resursa cerută nu a fost găsită.

4 DIN CE E FĂCUT UN SITE?

HTML = structura. CSS = aspectul. JavaScript = comportamentul interactiv. Browserul le combină într-o pagină vizibilă.

Un buton vizibil poate declanșa cod la clic.

ACTIVITATE — TEATRU FĂRĂ WI-FI

Un student este browserul, unul DNS, unul serverul. Cereți o pagină desenată pe hârtie.

O FRAZĂ DE ȚINUT MINTE

BROWSERUL este aplicația care CERE, PRIMEȘTE și AFIȘEAZĂ pagini.

Întrebare: Poți deschide un fișier HTML local fără internet?

TERMINALUL: O ALTĂ UȘĂ

Aceeași mașină: cu clicuri sau cu instrucțiuni scrise.

SCANEAZA

CUM SE ÎNTÂMPLĂ • URMĂREȘTE SĂGEȚILE

01 SCRII O COMANDĂ → 02 SHELL-UL O CITEȘTE → 03 OS EXECUTĂ → 04 VEZI RĂSPUNSUL

PATRU IDEI DE REȚINUT

1 CE ESTE TERMINALUL?

O aplicație unde tastezi comenzi. „Shell” este programul care interpretează comenzile. Nu sunt același lucru.

Fereastra = terminal; interpretul = shell.

2 WINDOWS

Start → caută „Terminal” sau „PowerShell”. În Terminal poți deschide PowerShell ori Command Prompt; comenzile pot diferi.

PowerShell: pwd (unde sunt?), ls (ce este aici?).

3 LINUX / macOS

Ubuntu: Ctrl+Alt+T sau caută „Terminal”. macOS: Spotlight → Terminal sau Applications → Utilities.

În terminal: pwd, ls, cd nume_folder.

4 DAR PE ANDROID?

Android are tot un OS și un shell, dar terminalul nu este o aplicație standard accesibilă tuturor. Dezvoltatorii folosesc instrumente speciale.

Din PC, ADB poate deschide un shell Android; nu e necesar acum.

ACTIVITATE — GHICEȘTE EFECTUL

Profesorul scrie „pwd” și „ls” pe ecran. Studenții prezic răspunsul înainte de Enter.

O FRAZĂ DE ȚINUT MINTE

GUI = îi ARĂȚI ce vrei prin clic. TERMINAL = îi SCRII ce vrei.

Întrebare: Care este diferența dintre Terminal și shell?

DE LA TERMINAL LA ECRAN

Un experiment demonstrabil la UN SINGUR calculator.

SCANEAZA

CUM SE ÎNTÂMPLĂ • URMĂREȘTE SĂGEȚILE

01 CREEZI UN FOLDER → 02 ÎL VEZI GRAFIC → 03 PUI O PAGINĂ → 04 O DESCHIZI ÎN BROWSER

PATRU IDEI DE REȚINUT

1 1. FĂ UN LUCRU VIZIBIL

Deschide Terminal/PowerShell. Tastează „mkdir demo”. Se creează folderul în locul curent. Tastează „cd demo”, apoi verifică-l și din File Explorer / Files.

„pwd” îți spune locul; „ls” arată conținutul.

2 2. SCRIE O PAGINĂ

În folderul demo, creează cu un editor text fișierul index.html. Scrie exact codul din caseta de mai jos. În Notepad, la salvare alege „All files”, nu .txt.

<h1>Salut, lume!</h1> → titlu mare în pagină.

3 3. FĂ PAGINA VIZIBILĂ

Deschide index.html în browser. Modifică „Salut” în „Bun venit”, salvează și reîncarcă pagina. Vezi rezultatul codului!

Fișierul local poate fi deschis și fără internet.

4 4. PASUL „SERVER” (OPȚIONAL)

Dacă Python este instalat, din folderul demo: pornește un server local și vizitează http://127.0.0.1:8000/. Oprește cu Ctrl+C.

Win: py -m http.server 8000 --bind 127.0.0.1
Linux/Mac: python3 -m http.server 8000 --bind 127.0.0.1

ACTIVITATE **DEMO PENTRU TOATĂ CLASA**

Înainte de fiecare Enter întreabă: „Ce se va schimba? Unde voi vedea efectul?”.

O FRAZĂ DE ȚINUT MINTE

TERMINALUL modifică fișiere; browserul poate transforma acele fișiere în pagini.

Atenție: serverul Python este doar pentru demonstrații locale, nu pentru producție.

CUM SE CITEȘTE CODUL

Cititul codului înseamnă urmărirea unor pași, nu memorarea limbajului.

SCANEAZA

CUM SE ÎNTÂMPLĂ • URMĂREȘTE SĂGEȚILE

01 CITEȘTE LINIA → 02 CALCULEAZĂ EXPRESIA → 03 URMĂREȘTE VALOAREA → 04 OBSERVĂ REZULTATUL

PATRU IDEI DE REȚINUT

1 VARIABILA = O ETICHETĂ

„scor = 3” atribuie valoarea 3 numelui scor. Mai târziu poți modifica valoarea. „=” atribuie; „==” verifică egalitatea.

scor = 3 → scor = scor + 1 → scor devine 4.

2 FUNCȚIA = O MINI-REȚETĂ

„len” calculează lungimea; „print” afișează. La apeluri imbricate rezolvi mai întâi rezultatul apelului interior.

print(len("Ana")) → len("Ana") = 3 → print(3).

3 ORDINEA NU E INVERSĂ!

Nu există regula generală „codul se citește de la dreapta la stânga”. Instrucțiunile se urmează în ordinea execuției; Python evaluează în general expresiile stânga-dreapta.

Doar apelul INTERIOR trebuie rezolvat înaintea celui exterior.

4 IF, FOR ȘI INDENTAREA

„if” alege un drum; „for” repetă. Spațiile de la începutul liniei formează blocurile de instrucțiuni din Python.

if scor > 3: → execută blocul doar dacă este adevărat.

ACTIVITATE — CITEȘTE FĂRĂ A RULA

Scrie: x = 2; x = x + 3; print(x). Cere tuturor să prezică valoarea afișată.

O FRAZĂ DE ȚINUT MINTE

CODUL este o succesiune de stări: „ce valoare există ACUM?”

Întrebare: Cât afișează print(len("cat"))?

ALGORITMI ȘI DEPANARE

Cum rezolvi sistematic o problemă chiar fără calculator.

SCANEAZA

CUM SE ÎNTÂMPLĂ • URMĂREȘTE SĂGEȚILE

01 PRIMEȘTI O CERINȚĂ → 02 ALEGI PAȘI CLARI → 03 EXECUȚI PE RÂND → 04 VERIFICI REZULTATUL

PATRU IDEI DE REȚINUT

1 ALGORITM = REȚETĂ PRECISĂ

O listă finită de instrucțiuni pentru o problemă. Intrarea este ce primești; ieșirea este ce obții. Aceiași pași trebuie să fie neambigui.

Exemplu: primești 3 numere → găsești cel mai mare.

2 DECIZIE ȘI REPETIȚIE

Decizie: „dacă plouă, ia umbrela”. Repetiție: „pentru fiecare carte, citește titlul”. Acestea sunt fundamentale.

„Dacă...” = condiție. „Pentru fiecare...” = buclă.

3 URMĂREȘTE O TABELĂ

Când codul modifică datele, notează starea după fiecare pas. Nu ghici rezultatul dintr-o privire.

pas 0: x=1 | pas 1: x=2 | pas 2: x=4.

4 ERORILE NU SUNT TOATE LA FEL

Sintaxă: instrucțiune scrisă greșit. Execuție: programul se oprește. Logică: rulează, dar răspunde greșit.

Depanare: reproduce → găsește primul pas greșit → verifică.

ACTIVITATE **ROBOTUL UMAN**

Spune unui coleg cum să deseneze un pătrat, fără să arăți. Ce instrucțiuni lipseau?

O FRAZĂ DE ȚINUT MINTE

Un program bun nu este „magie”: îl poți SIMULA pas cu pas.

Întrebare: Un rezultat incorect fără eroare este o eroare de... ?

STRUCTURI DE DATE — I

Datele au nevoie de organizare, la fel ca lucrurile din viața reală.

SCANEAZA

CUM SE ÎNTÂMPLĂ • URMĂREȘTE SĂGEȚILE

01 AI MULTE VALORI → 02 LE ORGANIZEZI → 03 ALEGI O REGULĂ → 04 LE FOLOSEȘTI RAPID

PATRU IDEI DE REȚINUT

1 LISTĂ = OBIECTE ÎN ORDINE

Ca locurile într-un tren: fiecare are o poziție. În Python, prima poziție este 0, nu 1. Poți parcurge sau modifica elemente.

["Ana", "Dan", "Mia"] → poziția 0 este „Ana”.

2 STIVĂ (STACK) = FARFURII

Adaugi sus și iei tot de sus. Ultimul pus este primul scos: LIFO. Utilă la „Undo” sau apeluri de funcții.

Pui A, apoi B, apoi C → scoți C prima dată.

3 COADĂ (QUEUE) = LA CASE

Primul venit este primul servit: FIFO. Elementele intră în spate și ies din față. Utilă la sarcini în așteptare.

Intră A, B, C → iese A prima dată.

4 MULȚIME (SET) = UNICI

Ține elemente fără duplicate. Ordinea nu este garantată. Bună pentru întrebarea „a mai apărut acest nume?”.

[A, B, A] într-o mulțime → {A, B}.

ACTIVITATE CLASĂ FĂRĂ ECRANE

Trei studenți sunt A, B, C. Arătați cu poziția corpului ordinea ieșirii din stivă și coadă.

O FRAZĂ DE ȚINUT MINTE

LISTĂ = poziție · STIVĂ = ultimul · COADĂ = primul · SET = fără duplicate.

Întrebare: Ce structură seamănă cu butonul „Undo”?

STRUCTURI DE DATE — II

Cum găsim rapid un obiect sau reprezentăm legături.

SCANEAZA

CUM SE ÎNTÂMPLĂ • URMĂREȘTE SĂGEȚILE

01 CE VREI SĂ ȘTII? → 02 ALEGI STRUCTURA → 03 FACI OPERAȚIA → 04 VERIFICI COSTUL

PATRU IDEI DE REȚINUT

1 DICȚIONAR = CUVÂNT → SENS

Cauți după o CHEIE, nu după poziție. Exemple: nume → telefon; produs → preț. În Python se numește dict.

{"Ana": 12, "Dan": 9} → cheia „Ana” dă 12.

2 ARBORE = FAMILIE DE RAMURI

Fiecare nod (în afară de rădăcină) are un părinte. Folderele dintr-un calculator seamănă cu un arbore.

Facultate → anul I → semestrul 1 → laborator.

3 GRAF = PUNCTE ȘI LEGĂTURI

Noduri conectate prin muchii. O rețea de prieteni sau stații de metrou este un graf. Pot exista cicluri.

A — B — C și A — C: există mai multe drumuri.

4 CUM ALEGI?

Ordine → listă. Primul sosit → coadă. Ultimul adăugat → stivă. Găsire după cheie → dict. Relații → graf.

Structura potrivită simplifică programul și poate economisi muncă.

ACTIVITATE

DESENEAZĂ LEGĂTURI

Pe hârtie: 5 orașe și drumuri între ele. Poți ajunge din orice oraș în oricare altul?

O FRAZĂ DE ȚINUT MINTE

STRUCTURA DE DATE este REGULA după care păstrezi și găsești informația.

Întrebare: Ce folosești pentru „nume student → notă”?

FIȘIERE, MEMORIE, DATE

Unde „stă” informația și de ce uneori dispare.

SCANEAZA

CUM SE ÎNTÂMPLĂ • URMĂREȘTE SĂGEȚILE

01 APLICAȚIA RULEAZĂ → 02 DATELE STAU ÎN RAM → 03 APEȘI „SAVE” → 04 FIȘIERUL RĂMÂNE

PATRU IDEI DE REȚINUT

1 RAM ≠ SSD / DISC

RAM ține datele de lucru, temporar. SSD/discul păstrează fișiere după oprire. „Save” scrie date pentru păstrare.

Un document nesalvat se poate pierde la oprire.

2 FIȘIER, FOLDER, CALE

Fișier = date salvate sub un nume. Folder = grupare de fișiere. Cale = adresa fișierului în foldere.

Ex.: Documente / curs / tema.txt.

3 BIT, BYTE, TEXT

Calculatorul stochează informația în biți (0 sau 1). 8 biți = 1 byte. Textul, imaginile și sunetul sunt codificate ca date.

„A” devine un număr după o regulă de codare.

4 EXTENSII ȘI COPII

Numele .txt, .jpg, .py sugerează tipul fișierului, dar nu garantează conținutul. Copiile de siguranță protejează munca.

„În cloud” înseamnă, de regulă, pe alt calculator (server).

ACTIVITATE — DESENEAZĂ O CALE

Pe hârtie: universitate → anul I → laborator → tema.py. Unde este fișierul și unde este folderul?

O FRAZĂ DE ȚINUT MINTE

RAM = masa de lucru · SSD = dulapul de păstrare · fișier = obiectul salvat.

Întrebare: Dacă oprești PC-ul fără să salvezi, ce riști să pierzi?

SIGURANȚĂ + AI CU CAP

Obiceiuri bune încă din prima zi de facultate.

SCANEAZA

CUM SE ÎNTÂMPLĂ • URMĂREȘTE SĂGEȚILE

01 OPREȘTE-TE → 02 VERIFICĂ SURSA → 03 LIMITĂ ACCESUL → 04 TESTEAZĂ REZULTATUL

PATRU IDEI DE REȚINUT

1 CONTURI PROTEJATE

Folosește parole unice și lungi cu un manager; preferă passkeys când există. Activează autentificarea multifactor (MFA).

Nu spune nimănui codurile MFA sau de recuperare.

2 LINKURI, FIȘIERE, QR

Un mesaj urgent poate fi fraudulos. Verifică domeniul real, expeditorul și destinația înainte de clic. Și codurile QR pot ascunde linkuri.

Un lacăt HTTPS nu garantează că site-ul este cinstit.

3 TERMINALUL POATE FACE RĂU

Nu lipi comenzi necunoscute și nu rula scripturi din surse neverificate. „Administrator/root” oferă puteri suplimentare; cere permisiuni doar dacă trebuie.

Nu dezactiva protecții doar pentru că „așa spune tutorialul”.

4 AI ÎȚI PROPUNE; TU VERIFICI

AI poate explica, dar poate inventa sau produce cod nesigur. Cere exemple, verifică rezultatul și nu trimite secrete sau date personale.

Orice soluție: înțelege → testează → verifică.

ACTIVITATE **SIMULARE ANTI-PHISHING**

„Contul tău expiră azi, intră imediat pe login-univ.example”. Ce verifici înainte să atingi linkul?

O FRAZĂ DE ȚINUT MINTE

ÎNTREABĂ-TE: „Înțeleg ce va face? Am verificat? Pot reveni?”

Întrebare: Este sigur să rulezi automat cod primit de la un AI?

NU TREBUIE SĂ ȘTII MATEMATICĂ SAU PROGRAMARE

BIG-O

SE CITEȘTE „O MARE”

SCANEAZA

CÂT CREȘTE MUNCA atunci când ai MAI MULTE lucruri?

4 OBIECTE → 8 OBIECTE **Le dublăm.**

UN PAS
o acțiune simplă pe care o faci

n
câte lucruri ai de verificat

O(...)
cum crește numărul de pași

ACELEAȘI 4 → 8 OBIECTE. ȘASE FELURI ÎN CARE POATE CREȘTE MUNCA:

O(1) NU CREȘTE

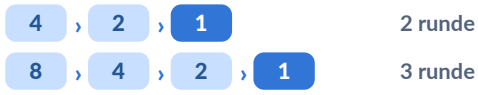
Știu exact sertarul. Îl deschid direct, oricâte sertare ar exista.



4 obiecte **1 pas** → 8 obiecte **1 pas**

O(log n) CREȘTE FOARTE PUȚIN

Caut într-o listă ORDONATĂ. Elimin jumătate la fiecare rundă.



4 obiecte **2 runde** → 8 obiecte **3 runde**

O(n) CREȘTE DE 2 ORI

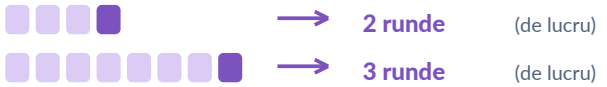
Caut prin sertare PE RÂND. Dacă e ultimul, le verific pe toate.



4 obiecte **4 verificări** → 8 obiecte **8 verificări**

O(n log n) CREȘTE PUȚIN PESTE 2 ORI

Ordonez cartonașe: le împart în grupuri și le reunesc ordonat.



4 obiecte **~8 pași*** → 8 obiecte **~24 pași***

O(n²) CREȘTE DE 4 ORI

Fiecare obiect este comparat cu TOATE obiectele (chiar și cu el însuși).



4 obiecte **16 comp.** → 8 obiecte **64 comp.**

O(2ⁿ) CREȘTE ENORM

Pentru fiecare obiect aleg DA sau NU. Încerc TOATE combinațiile posibile.



4 obiecte **16 var.** → 8 obiecte **256 var.**

ȚINE MINTE O SINGURĂ ÎNTREBARE:

„Dacă dublez numărul de obiecte, CÂT CREȘTE MUNCA?”

la fel · puțin · ×2 · peste ×2 · ×4 · exploziv

Big-O nu dă secunde sau pași exacți: arată o limită pentru cum poate crește munca.

* La O(n log n), numerele ilustrează un model simplificat, nu pași măsurați.

FIȘĂ DE RECUNOAȘTERE • ANUL I

BIG -O

CÂT DE REPEDE CREȘTE MUNCA ATUNCI CÂND CRESC DATELE?

n = numărul de elemente



SCANEAZA

EXPERIMENTUL MENTAL

10 → 20

DUBLEZI DATELE (n).

Ce se întâmplă cu numărul de operații?

Compară valorile funcțiilor-model din carduri.

CELE 6 TIPARE PE CARE TREBUIE SĂ LE RECUNOȘTI

MAI BINE → MAI GREU DE SCALAT

O(1)

CONSTANT

Un acces. Nu depinde de n.

```
a[0]
```

1 → 1

O(log n)

LOGARITMIC

Tai problema în jumătate.

```
n //= 2
```

3,3 → 4,3

O(n)

LINIAR

Treci o dată prin toate.

```
for x in a:
```

10 → 20

O(n log n)

N × LOG N

Împarți, apoi combini.

```
merge_sort(a)
```

33 → 86

O(n^2)

PĂTRATIC

Compari fiecare cu fiecare.

2 bucle imbricate

100 → 400

O(2^n)

EXPONENȚIAL

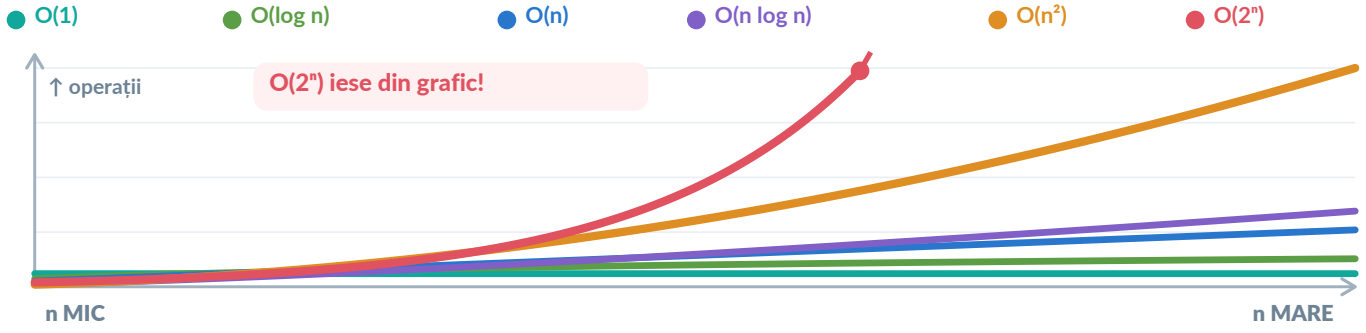
Încerci toate submulțimile.

toate submulțimile

1 024 → 1 048 576

CURBELE: CÂND n CREȘTE →

forma contează, nu secundele



3 REGULI DE SUPRAVIEȚUIRE

01 BUCLE UNA DUPĂ ALTA

$O(n) + O(n) = O(n)$

aduni; păstrezi termenul dominant

02 BUCLE UNA ÎN ALTA

$O(n) \times O(n) = O(n^2)$

înmulțești numărul de repetări

03 IGNORI CONSTANTELE

$O(3n^2 + 7n + 2) = O(n^2)$

pentru n mare, domină n^2

DE ȚINUT MINTE

Big-O = limită superioară de creștere, nu timp în secunde.

O(1) ≠ un singur pas. Precizează cazul analizat și resursa: timp / memorie.